অধ্যায় ১০

১০

Asynchronous JavaScript: Callbacks, Promises, async/await এবং Event Loop

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Synchronous ও Asynchronous কাজের পার্থক্য, এবং দীর্ঘ কাজ কেন Page আটকে দেয়, তা ব্যাখ্যা করতে।
- Call Stack, Task Queue, Microtask Queue এবং Event Loop দিয়ে কোড কোন ক্রমে চলবে তা আগে থেকে বলতে।
- Callback, Promise এবং `async` / `await` দিয়ে অপেক্ষার কাজ লিখতে, এবং তিনটির সম্পর্ক বুঝতে।
- `try` / `catch` / `finally` দিয়ে Asynchronous ত্রুটি সামলাতে।
- `fetch` দিয়ে তথ্য আনতে, এবং HTTP ত্রুটি আলাদাভাবে ধরতে।
- `Promise.all`, `allSettled`, `race` ও `any` দিয়ে একাধিক কাজ একসঙ্গে চালাতে।
- `AbortController` দিয়ে কাজ বাতিল করতে, সময়সীমা বসাতে এবং Race Condition এড়াতে।
- সাময়িক ত্রুটিতে বিরতি বাড়িয়ে আবার চেষ্টা (Retry ও Backoff) করতে।
- একটি সম্পূর্ণ **User Directory API Explorer** তৈরি করতে, এবং কৃত্রিম Server দিয়ে তার Loading, Error ও Race-এর আচরণ পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৫: Function, Arrow Function এবং Closure।
- অধ্যায় ৭: Object, Destructuring এবং JSON।
- অধ্যায় ৯: DOM, Event Listener এবং `render`-ভিত্তিক গঠন।
- Node.js 22 বা নতুন সংস্করণ, এবং একটি আধুনিক Browser।
- `chapter-10` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

এ পর্যন্ত প্রতিটি লাইন আগের লাইন শেষ হওয়ার সঙ্গে সঙ্গে চলেছে। কিন্তু বাস্তব Program-এর অনেক কাজে অপেক্ষা করতে হয়: Server থেকে তথ্য আসতে কয়েকশো মিলিসেকেন্ড, একটি File পড়তে কিছুক্ষণ, ব্যবহারকারী কখন Click করবেন তার ঠিক নেই। এই অপেক্ষার সময়ে Program থেমে থাকলে Page জমে যায়: Button কাজ করে না, লেখা যায় না, Scroll হয় না।

JavaScript এর সমাধান করে অপেক্ষার কাজকে পেছনে পাঠিয়ে, এবং কাজ শেষ হলে ফল নিয়ে ফিরে এসে। এই ধারণাকে বলে **Asynchronous Programming**। আধুনিক Web Application-এর প্রায় প্রতিটি অংশ এর ওপর নির্ভর করে, এবং এর ভুলগুলো সবচেয়ে কঠিন Bug-গুলোর মধ্যে পড়ে: মাঝে মাঝে ঘটে, পরীক্ষার সময় ঘটে না। এই অধ্যায়ে আপনি শুধু Syntax নয়, ভেতরের নিয়মটি শিখবেন, যাতে Program কোন ক্রমে চলবে তা আপনি নিজেই বলতে পারেন।

১০.১ Synchronous এবং Asynchronous

একটি রেস্তোরাঁর কথা ভাবুন। একজন পরিবেশনকর্মী অর্ডার নিয়ে রান্নাঘরে দেন, তারপর রান্না শেষ হওয়া পর্যন্ত দাঁড়িয়ে না থেকে পরের টেবিলে যান। রান্না হলে রান্নাঘর তাঁকে ডাকে। JavaScript ঠিক এভাবে কাজ করে: একজন কর্মী (একটি Thread), কিন্তু অপেক্ষার কাজ অন্যকে দিয়ে নিজে এগিয়ে যান।

restaurant.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('১. অর্ডার নেওয়া হলো');

setTimeout(() => {
  console.log('৩. খাবার তৈরি');
}, 0);

console.log('২. পরের অর্ডার');
```

Output

```
১. অর্ডার নেওয়া হলো
২. পরের অর্ডার
৩. খাবার তৈরি
```

`setTimeout(callback, 0)` মানে "এখনই" নয়; মানে "যত তাড়াতাড়ি সম্ভব, তবে এখনকার কাজ শেষ হওয়ার পরে"। এখনকার কাজ, অর্থাৎ পুরো File-টি চলা শেষ না হওয়া পর্যন্ত Callback অপেক্ষা করে।

JavaScript একসময়ে একটিই কাজ করে, তাই একটি দীর্ঘ হিসাব সব কিছু আটকে দেয়:

blocking.js · JavaScript চলবে: Browser Console ও Node.js

```
const start = Date.now();

setTimeout(() => {
  const late = Date.now() - start >= 200 ? 'কমপক্ষে 200 ms' : 'কম';
  console.log(`Timer চলল, দেরি ${late}`);
}, 0);

while (Date.now() - start < 200) {
  // 200 ms ধরে শুধু ঘুরছে
}

console.log('Loop শেষ');
```

Output

```
Loop শেষ
Timer চলল, দেরি কমপক্ষে 200 ms
```

Timer-এর সময় `0` হলেও সেটি চলেছে Loop শেষ হওয়ার পরে। Browser-এ এমন একটি Loop চলার সময় Page-এর কোনো Click বা Scroll-ও কাজ করে না। শিক্ষা: Network বা Timer-এর মতো অপেক্ষা JavaScript-কে আটকায় না, কিন্তু JavaScript-এর নিজের দীর্ঘ হিসাব আটকায়।

১০.২ Event Loop: ভেতরের নিয়ম

Program কোন ক্রমে চলবে তা বুঝতে চারটি অংশ জানা দরকার:

অংশ	কাজ
Call Stack	এই মুহূর্তে যে Function-গুলো চলছে; একটির ভেতরে আরেকটি ডাকলে ওপরে জমে
Host-এর সুবিধা	Timer, Network, Event; Browser বা Node.js নিজে এগুলো সামলায়, JavaScript-এর বাইরে
Task Queue	Timer শেষ, Click, Network-এর সাড়া—চলার জন্য অপেক্ষমাণ Callback
Microtask Queue	Promise-এর ফল ধরার Callback (`then` , `await` -এর পরের অংশ)

Event Loop-এর নিয়ম মোটামুটি এমন:

১. Call Stack-এর বর্তমান কাজ শেষ পর্যন্ত চলে।

২. Stack খালি হলে Microtask Queue-এর **সব** Microtask চলে; চলার সময় নতুন Microtask এলে সেটিও।

৩. তারপর Task Queue থেকে **একটি** Task নিয়ে আবার ধাপ ১। Browser-এ এর ফাঁকে পর্দা নতুন করে আঁকা হতে পারে।

মূল কথা: **Promise-এর Callback সব সময় Timer-এর Callback-এর আগে চলে,** যদিও দুটিই "পরে" চলার কথা। নিচের Code-এর Output আগে নিজে ভেবে বলুন, তারপর মিলিয়ে দেখুন।

event-loop.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('A: শুরু');

setTimeout(() => console.log('F: setTimeout'), 0);

Promise.resolve().then(() => console.log('D: Promise'));

async function task() {
  console.log('B: async Function, await-এর আগে');
  await null;
  console.log('E: await-এর পরে');
}

task();
console.log('C: শেষ লাইন');
```

Output

```
A: শুরু
B: async Function, await-এর আগে
C: শেষ লাইন
D: Promise
E: await-এর পরে
F: setTimeout
```

- **A, B, C** Synchronous: `task()` ডাকার সঙ্গে সঙ্গে তার প্রথম অংশ চলে; `async` Function `await` পর্যন্ত সাধারণ Function-এর মতোই।
- **D, E** Microtask: `then`-এর Callback এবং `await`-এর পরের অংশ। যে আগে Queue-তে ঢুকেছে, সে আগে।
- **F** Task: সব Microtask শেষ হওয়ার পর।

> **নোট:** এটি একটি সরলীকৃত মডেল। Browser ও Node.js-এর আসল Scheduler-এ আরও কয়েকটি Queue আছে, কিন্তু দৈনন্দিন Code-এর ক্রম বোঝার জন্য এই মডেলই যথেষ্ট।

১০.৩ Callback: প্রথম পদ্ধতি

অপেক্ষার কাজ শেষ হলে কী করতে হবে, তা একটি Function হিসেবে দিয়ে দেওয়া—এটিই **Callback**। Node.js-এর পুরোনো রীতিতে Callback-এর প্রথম Argument ত্রুটি, দ্বিতীয়টি ফল (Error-first Callback)।

callbacks.js · JavaScript চলবে: Browser Console ও Node.js

```
function loadProfile(id, callback) {
  setTimeout(() => {
    if (id <= 0) {
      callback(new Error('ভুল ID'));
      return;
    }
    callback(null, { id, name: 'Sara' });
  }, 100);
}

loadProfile(7, (error, profile) => {
  if (error) {
    console.log(`ব্যর্থ: ${error.message}`);
    return;
  }
  console.log(`পাওয়া গেল: ${profile.name}`);
});

loadProfile(0, (error) => {
  console.log(`ব্যর্থ: ${error.message}`);
});

console.log('অপেক্ষা চলছে...');
```

Output

```
অপেক্ষা চলছে...
পাওয়া গেল: Sara
ব্যর্থ: ভুল ID
```

সব Callback Asynchronous নয়: `array.map(callback)` সঙ্গে সঙ্গে চলে। Callback শুধু "পরে ডাকার জন্য দেওয়া Function"; কখন ডাকা হবে তা নির্ভর করে যে Function সেটি নিচ্ছে তার ওপর।

সমস্যা দেখা দেয় যখন একটির পর একটি অপেক্ষার কাজ লাগে:

nesting.js · JavaScript চলবে: Browser Console ও Node.js

```
function step(name, callback) {
  setTimeout(() => callback(null, name), 50);
}

step('User', (error1, user) => {
  step(`${user}-এর Order`, (error2, orders) => {
    step(`${orders}-এর Invoice`, (error3, invoice) => {
      console.log(invoice);
    });
  });
});
```

Output

```
User-এর Order-এর Invoice
```

প্রতিটি ধাপ আগের ধাপের ভেতরে ঢুকছে, Code ডান দিকে সরছে, এবং তিনটি `error`-এর একটিও যাচাই হয়নি। প্রতিটি স্তরে যাচাই যোগ করলে Code দ্বিগুণ হতো। এই অবস্থার প্রচলিত নাম **Callback Hell**। Promise এর সমাধান হিসেবে এসেছে।

১০.৪ Promise: ভবিষ্যতের ফলের প্রতিশ্রুতি

Promise একটি Object, যা একটি অপেক্ষার কাজের ফলের প্রতিনিধিত্ব করে। এর তিনটি অবস্থা:

অবস্থা	অর্থ
pending	কাজ চলছে
fulfilled	সফলভাবে শেষ, একটি মান পাওয়া গেছে
rejected	ব্যর্থ, একটি ত্রুটি পাওয়া গেছে

একবার fulfilled বা rejected হলে Promise আর কখনো অবস্থা বদলায় না। ফল ধরতে `then`, ত্রুটি ধরতে `catch`, আর দুই ক্ষেত্রেই শেষে কিছু করতে `finally`।

promise-basics.js · JavaScript চলবে: Browser Console ও Node.js

```
function wait(ms, value) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(value), ms);
  });
}

function failAfter(ms, message) {
  return new Promise((resolve, reject) => {
```

```
    setTimeout(() => reject(new Error(message)), ms);
  });
}

wait(100, 'Sara')
  .then((name) => name.toUpperCase())
  .then((upper) => console.log(`ফল: ${upper}`))
  .catch((error) => console.log(`ত্রুটি: ${error.message}`))
  .finally(() => console.log('প্রথম কাজ শেষ'));

failAfter(200, 'Server বন্ধ')
  .then(() => console.log('এটি চলবে না'))
  .catch((error) => console.log(`ত্রুটি: ${error.message}`))
  .finally(() => console.log('দ্বিতীয় কাজ শেষ'));
```

Output

```
ফল: SARA
প্রথম কাজ শেষ
ত্রুটি: Server বন্ধ
দ্বিতীয় কাজ শেষ
```

- `new Promise((resolve, reject) => { ... })` একটি Promise তৈরি করে। ভেতরের Function সঙ্গে সঙ্গে চলে; কাজ শেষে `resolve(value)` বা `reject(error)` ডাকে।
- **প্রতিটি `then` একটি নতুন Promise ফেরত দেয়,** তাই Chain করা যায়। একটি `then` যা ফেরত দেয়, পরের `then` তা পায়। ফেরত মান নিজে Promise হলে পরের `then` সেটি শেষ হওয়া পর্যন্ত অপেক্ষা করে।
- **একটি `catch` আগের সব ধাপের ত্রুটি ধরে।** দ্বিতীয় Chain-এ প্রথম `then` বাদ পড়েছে, সরাসরি `catch` চলেছে।

আগের Callback Hell-এর উদাহরণ Promise দিয়ে লিখলে ধাপগুলো সমান্তরালে নিচে নামে, এবং একটি `catch` সব ধাপের ত্রুটি ধরে:

promise-chain.js · JavaScript — চলবে: Browser Console ও Node.js

```
const step = (text) =>
  new Promise((resolve) => setTimeout(() => resolve(text), 50));

step('User')
  .then((user) => step(`${user}-এর Order`))
  .then((orders) => step(`${orders}-এর Invoice`))
  .then((invoice) => console.log(invoice))
  .catch((error) => console.log(`ত্রুটি: ${error.message}`));
```

Output

```
User-এর Order-এর Invoice
```

১০.৫ async এবং await

`async` / `await` Promise-এর ওপর তৈরি একটি লেখার ধরন, যাতে Asynchronous Code দেখতে Synchronous Code-এর মতো হয়।

async-await.js · JavaScript — চলবে: Browser Console ও Node.js

```
const step = (text) =>
  new Promise((resolve) => setTimeout(() => resolve(text), 50));

async function buildInvoice() {
  const user = await step('User');
  const orders = await step(`${user}-এর Order`);
  return step(`${orders}-এর Invoice`);
}

async function main() {
  try {
    const invoice = await buildInvoice();
    console.log(invoice);
  } catch (error) {
    console.log(`ত্রুটি: ${error.message}`);
  }
}

main();
console.log('main() ডাকা হলো; ফল পরে আসবে');
```

Output

```
main() ডাকা হলো; ফল পরে আসবে
User-এর Order-এর Invoice
```

- `await promise` Promise শেষ হওয়া পর্যন্ত **এই Function-কে** থামায়, এবং ফল দেয়। Promise rejected হলে `await` সেখানে Error ছোড়ে, যা সাধারণ `try` / `catch` ধরে।
- **পুরো Program থামে না।** শেষ লাইনটি আগে ছাপা হয়েছে: `main` যখন অপেক্ষা করছিল, বাকি Code এগিয়ে গেছে।
- `await` **শুধু** `async` **Function-এর ভেতরে** (এবং Module-এর সবচেয়ে ওপরের স্তরে, যা অধ্যায় ১২-এ আসবে)।

একটি `async` Function সব সময় Promise ফেরত দেয়, `return`-এ যা-ই লেখা থাকুক:

async-returns.js · JavaScript চলবে: Node.js

```
async function getAnswer() {
  return 42;
}

const result = getAnswer();
console.log(result);
console.log(result instanceof Promise);
result.then((value) => console.log(value));
```

Output

```
Promise { 42 }
true
42
```

তাই `async` Function-এর ফল চাইলে, ডাকার জায়গায় আবার `await` বা `then` লাগে। (`Promise { 42 }` লেখাটি Node.js-এর দেখানোর ধরন; Browser Console Promise-কে অন্যভাবে দেখায়।)

১০.৬ fetch: Server থেকে তথ্য আনা

`fetch(url)` Browser ও Node.js-এ Network থেকে তথ্য আনে, এবং একটি Promise ফেরত দেয়, যা একটি **Response** Object-এ শেষ হয়। Response-এর মূল অংশ (Body) পড়তে আরেকটি অপেক্ষা লাগে: `response.json()` বা `response.text()`।

fetch-basics.js · JavaScript চলবে: Browser Console ও Node.js

```
async function readJson(url) {
  const response = await fetch(url);
  console.log(`Status: ${response.status}, ok: ${response.ok}`);
  return response.json();
}

const url = 'data:application/json,{"name":"Sara","city":"Dhaka"}';

readJson(url).then((user) => console.log(`${user.name}, ${user.city}`));
```

Output

```
Status: 200, ok: true
Sara, Dhaka
```

উদাহরণটি Internet ছাড়াই চলে, কারণ `data:` ঠিকানায় তথ্য ঠিকানার ভেতরেই লেখা থাকে। একটি আসল Server-এর ক্ষেত্রে শুধু ঠিকানা বদলায়। নিচের Code-টি Browser-এর Console-এ চালিয়ে দেখতে পারেন; এর জন্য Internet সংযোগ লাগবে, এবং ফল Server-এর ওপর নির্ভর করে বলে বইয়ে তা ছাপা হয়নি:

real-api.js · JavaScript — চলবে: Browser

```
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then((response) => response.json())
  .then((user) => console.log(user.name, user.email));
```

HTTP ত্রুটি আলাদা

একটি গুরুত্বপূর্ণ ফাঁদ: Server `404` (পাওয়া যায়নি) বা `500` (Server-এ সমস্যা) পাঠালেও `fetch`-এর Promise **fulfilled** হয়। `fetch` শুধু তখনই rejected হয় যখন সাড়াই আসেনি: Network নেই, ঠিকানা ভুল, বা কাজ বাতিল। HTTP ত্রুটি নিজে যাচাই করতে হয় `response.ok` দিয়ে, যা Status 200 থেকে 299-এর মধ্যে হলে `true`।

http-errors.js · JavaScript — চলবে: Browser Console ও Node.js

```
// পরীক্ষার জন্য একটি কৃত্রিম fetch: আসল Network ছাড়াই নির্দিষ্ট সাড়া দেয়।
const fakeFetch = async (url) =>
  url.endsWith('/users')
    ? new Response('[{"id":1},{"id":2}]', { status: 200 })
    : new Response('Not Found', { status: 404 });

async function getJson(url) {
  const response = await fakeFetch(url);

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return response.json();
}

async function main() {
  for (const url of ['/api/users', '/api/userz']) {
    try {
      const data = await getJson(url);
      console.log(`${url}: ${data.length}টি User`);
    } catch (error) {
      console.log(`${url}: ব্যর্থ, ${error.message}`);
    }
  }
}

main();
```

Output

```
/api/users: 2টি User
/api/userz: ব্যর্থ, HTTP 404
```

এখানে একটি পেশাদার কৌশলও দেখা গেল: `fetch`-এর জায়গায় একটি **কৃত্রিম fetch**, যা সব সময় একই সাড়া দেয়। আসল Server-এর সাড়া প্রতিবার আলাদা হতে পারে, Server বন্ধও থাকতে পারে; কৃত্রিম সাড়া দিয়ে Test সব সময় একই ফল দেয়। Project-এর Test এই কৌশলেই লেখা।

১০.৭ একাধিক কাজ: একটির পর একটি, নাকি একসঙ্গে

দুটি কাজ যদি একটি অন্যটির ফলের ওপর নির্ভর না করে, তাহলে একটির পর একটি `await` করা সময়ের অপচয়।

sequential-vs-parallel.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));
const rounded = (start) => Math.round((Date.now() - start) / 100) * 100;

async function main() {
  let start = Date.now();
  const a = await wait(200, 'User');
  const b = await wait(200, 'Order');
  console.log(`একটির পর একটি: ${a}, ${b}; প্রায় ${rounded(start)} ms`);

  start = Date.now();
  const [c, d] = await Promise.all([wait(200, 'User'), wait(200, 'Order')]);
  console.log(`একসঙ্গে: ${c}, ${d}; প্রায় ${rounded(start)} ms`);
}

main();
```

Output

```
একটির পর একটি: User, Order; প্রায় 400 ms
একসঙ্গে: User, Order; প্রায় 200 ms
```

`Promise.all` দুটি কাজই একসঙ্গে শুরু করে এবং দুটির শেষ হওয়ার জন্য অপেক্ষা করে; ফলগুলো আসে দেওয়া ক্রমে, শেষ হওয়ার ক্রমে নয়। কিন্তু কাজ যদি নির্ভরশীল হয় (Order আনতে User-এর ID লাগে), তাহলে একটির পর একটিই একমাত্র উপায়।

একসঙ্গে চালানোর চারটি উপায়, ব্যর্থতার ক্ষেত্রে আলাদা আচরণ:

combinators.js · JavaScript চলবে: Browser Console ও Node.js

```
const ok = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));
const fail = (ms, message) =>
  new Promise((_, reject) => setTimeout(() => reject(new Error(message)), ms));

async function main() {
  try {
    await Promise.all([ok(50, 'ক'), fail(100, 'খ ব্যর্থ'), ok(150, 'গ')]);
  } catch (error) {
    console.log(`all: ${error.message}`);
  }

  const settled = await Promise.allSettled([ok(50, 'ক'), fail(100, 'খ ব্যর্থ')]);
  const report = settled.map((result) =>
    result.status === 'fulfilled' ? result.value : result.reason.message);
  console.log(`allSettled: ${report.join(' | ')}`);

  const fastest = await Promise.race([ok(150, 'ধীর'), ok(50, 'দ্রুত')]);
  console.log(`race: ${fastest}`);

  const firstOk = await Promise.any([fail(50, 'x'), ok(100, 'প্রথম সফল')]);
  console.log(`any: ${firstOk}`);
}

main();
```

Output

```
all: খ ব্যর্থ
allSettled: ক | খ ব্যর্থ
race: দ্রুত
any: প্রথম সফল
```

পদ্ধতি	কখন শেষ	কখন ব্যবহার
`Promise.all`	সব সফল হলে; একটি ব্যর্থ হলেই ব্যর্থ	সবগুলো ছাড়া কাজ এগোয় না
`Promise.allSettled`	সব শেষ হলে, সফল বা ব্যর্থ	প্রতিটির ফলাফল আলাদাভাবে জানাতে
`Promise.race`	প্রথম যেটি শেষ হয়, সফল বা ব্যর্থ	সময়সীমা বসাতে
`Promise.any`	প্রথম সফলটি; সব ব্যর্থ হলে ব্যর্থ	একাধিক উৎসের যেকোনো একটি যথেষ্ট

১০.৮ বাতিল করা, সময়সীমা এবং Race Condition

ব্যবহারকারী একটি Page ছেড়ে চলে গেলেন, বা নতুন কিছু খুঁজলেন—পুরোনো Request-এর আর দরকার নেই। `AbortController` দিয়ে চলমান কাজ বাতিল করা যায়। Controller-এর `signal` কাজটিকে দেওয়া হয়; `controller.abort()` ডাকলে কাজটি একটি `AbortError` দিয়ে rejected হয়। `fetch(url, { signal })` এটি সরাসরি বোঝে।

abort.js · JavaScript — চলবে: Browser Console ও Node.js

```
function wait(ms, signal) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => resolve(`${ms} ms পরে শেষ`), ms);
    signal.addEventListener('abort', () => {
      clearTimeout(timer);
      reject(signal.reason);
    });
  });
}

const controller = new AbortController();

wait(500, controller.signal)
  .then((text) => console.log(text))
  .catch((error) => console.log(`বাতিল: ${error.name}`));

setTimeout(() => controller.abort(), 100);

wait(500, AbortSignal.timeout(150))
  .then((text) => console.log(text))
  .catch((error) => console.log(`সময় শেষ: ${error.name}`));
```

Output

```
বাতিল: AbortError
সময় শেষ: TimeoutError
```

`AbortSignal.timeout(ms)` নির্দিষ্ট সময় পরে নিজে নিজেই বাতিল হয়, এবং ত্রুটির নাম হয় `TimeoutError`। একটি Server সাড়া না দিলে ব্যবহারকারীকে অনির্দিষ্টকাল অপেক্ষা করানো উচিত নয়; প্রতিটি Network Request-এ একটি সময়সীমা থাকা উচিত।

Race Condition

ব্যবহারকারী Search Box-এ "Ja" লিখলেন, একটি Request গেল। তারপর "JavaScript" লিখলেন, আরেকটি Request গেল। কোনো কারণে প্রথম Request-এর সাড়া দেরিতে এলো। কোন ফল পর্দায় থাকবে?

race-condition.js · JavaScript চলবে: Browser Console ও Node.js

```
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

// পুরোনো খোঁজার সাড়া ধীরে আসে, নতুনটির দ্রুত।
const searchServer = (query) =>
  wait(query === 'Ja' ? 300 : 100, `"${query}"-এর ফল`);

let shown = '';
let latest = 0;

async function onType(query, guard) {
  const ticket = ++latest;
  const result = await searchServer(query);

  if (guard && ticket !== latest) {
    console.log(`পুরোনো ফল বাদ: ${result}`);
    return;
  }
  shown = result;
}

async function demo(guard) {
  await Promise.all([onType('Ja', guard), onType('JavaScript', guard)]);
  console.log(`${guard ? 'Guard সহ' : 'Guard ছাড়া'}: পর্দায় ${shown}`);
}

demo(false).then(() => demo(true));
```

Output

```
Guard ছাড়া: পর্দায় "Ja"-এর ফল
পুরোনো ফল বাদ: "Ja"-এর ফল
Guard সহ: পর্দায় "JavaScript"-এর ফল
```

Guard ছাড়া, যে সাড়া **শেষে এসেছে** সেটি পর্দায় থেকে গেছে, যদিও তা পুরোনো খোঁজার। দুটি কাজ কোন ক্রমে শেষ হবে তার ওপর ফল নির্ভর করছে—এটিই **Race Condition**। এই Bug সাধারণত দ্রুত Network-এ ধরা পড়ে না, আর ধীর Mobile Network-এ ব্যবহারকারীর কাছে ধরা পড়ে।

সমাধান দুটি, এবং Project দুটোই ব্যবহার করে: প্রতিটি Request-এ একটি ক্রমিক নম্বর দিয়ে শুধু সর্বশেষটির ফল গ্রহণ করা (ওপরের `ticket`), এবং নতুন Request শুরুর আগে পুরোনোটি `AbortController` দিয়ে বাতিল করা, যাতে অপ্রয়োজনীয় Network-এর কাজও থামে।

১০.৯ Retry, Timeout এবং Backoff

কিছু ত্রুটি সাময়িক: Server এক মুহূর্তের জন্য ব্যস্ত, Network ক্ষণিকের জন্য বিচ্ছিন্ন। এসব ক্ষেত্রে একটু থেমে আবার চেষ্টা করলে প্রায়ই সফল হয়। প্রতিবার বিরতি দ্বিগুণ করাকে বলে **Exponential Backoff**: এতে সমস্যায় থাকা Server-এ চাপ কমে।

retry.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

let calls = 0;
async function flakyServer() {
  calls += 1;
  if (calls < 3) {
    throw new Error(`চেষ্টা ${calls}: Server ব্যস্ত`);
  }
  return 'তথ্য পাওয়া গেছে';
}

async function withRetry(task, retries = 3, baseDelay = 100) {
  for (let attempt = 0; ; attempt += 1) {
    try {
      return await task();
    } catch (error) {
      if (attempt >= retries) {
        throw error;
      }
      const delay = baseDelay * 2 ** attempt;
      console.log(`${error.message}; ${delay} ms পরে আবার`);
      await wait(delay);
    }
  }
}

withRetry(flakyServer).then((data) => console.log(data));
```

Output

```
চেষ্টা 1: Server ব্যস্ত; 100 ms পরে আবার
চেষ্টা 2: Server ব্যস্ত; 200 ms পরে আবার
তথ্য পাওয়া গেছে
```

- `return await task()` : এখানে `await` অপরিহার্য। শুধু `return task()` লিখলে Promise-টি `try` থেকে বেরিয়ে যেত, এবং তার ব্যর্থতা এই `catch` ধরত না।
- **সব ত্রুটিতে আবার চেষ্টা নয়।** `404` (ঠিকানা নেই) বা `400` (ভুল Request) আবার পাঠালেও একই ফল আসবে। Retry শুধু সাময়িক ত্রুটিতে: Network বিচ্ছিন্ন, সময়সীমা পার, বা `5xx` Status। ব্যবহারকারী নিজে বাতিল করলে

কখনোই নয়।

- **সীমা রাখুন:** চেষ্টার সংখ্যা সীমিত, এবং মোট অপেক্ষা যুক্তিসঙ্গত।

১০.১০ সাধারণ ভুল

সবচেয়ে প্রচলিত ভুল `await` ভুলে যাওয়া:

forgot-await.js · JavaScript — চলবে: Node.js

```
async function getUser() {
  return { name: 'Sara' };
}

const user = getUser();
console.log(user.name);
console.log(user);
```

Output

```
undefined
Promise { { name: 'Sara' } }
```

`user` আসলে একটি Promise, User নয়; তাই `user.name` হলো `undefined`। কোনো Error হয় না, যা ভুলটিকে খুঁজে পাওয়া কঠিন করে।

দ্বিতীয়টি: `forEach`-এর ভেতরে `await`। `forEach` Callback-এর Promise-এর জন্য অপেক্ষা করে না।

foreach-await.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

async function withForEach(ids) {
  const names = [];
  ids.forEach(async (id) => {
    names.push(await wait(50, `User ${id}`));
  });
  return names.length;
}

async function withForOf(ids) {
  const names = [];
  for (const id of ids) {
    names.push(await wait(50, `User ${id}`));
  }
  return names.length;
```

```
}

withForEach([1, 2, 3]).then((count) => console.log(`forEach: ${count}টি`));
withForOf([1, 2, 3]).then((count) => console.log(`for...of: ${count}টি`));
```

Output

```
forEach: 0টি
for...of: 3টি
```

তৃতীয়টি: ধরা না পড়া Rejection। একটি rejected Promise-এর কোনো `catch` না থাকলে Browser Console-এ "Uncaught (in promise)" দেখায়, আর Node.js পুরো Program বন্ধ করে দেয়।

unhandled.js · JavaScript — চলবে: Node.js

```
async function save() {
  throw new Error('Disk ভর্তি');
}

save();
console.log('এই লাইন চলে');
```

Error Message

```
এই লাইন চলে
Error: Disk ভর্তি
```

ভুল	কী দেখা যায়	সমাধান
`await` ভুলে যাওয়া	`undefined`, বা `[object Promise]` লেখা	`async` Function-এর ফলে `await`
`async` ছাড়া Function-এ `await`	`SyntaxError`	Function-কে `async` করুন
`forEach`-এ `await`	কাজ শেষের আগেই পরের লাইন	`for...of`, অথবা `Promise.all` ও `map`
স্বাধীন কাজ একটির পর একটি `await`	অকারণে ধীর	`Promise.all`
`fetch`-এর পর `response.ok` যাচাই না করা	`404`-এর লেখা JSON হিসেবে পড়তে গিয়ে বিভ্রান্তিকর Error	`if (!response.ok) throw ...`
`catch` নেই	Uncaught (in promise); Node.js বন্ধ	প্রতিটি Chain-এর শেষে `catch`, বা `try` / `catch`

ভুল	কী দেখা যায়	সমাধান
`try`-এর ভেতরে `return promise`	ত্রুটি `catch`-এ ধরা পড়ে না	`return await promise`
সময়সীমা নেই	Loading অনির্দিষ্টকাল ঘোরে	`AbortSignal.timeout()`
পুরোনো সাড়া নতুনটির ওপরে	ভুল ফল পর্দায়	ক্রমিক নম্বর, `AbortController`
সব ত্রুটিতে Retry	`404`-এ অকারণে বারবার চেষ্টা	শুধু সাময়িক ত্রুটিতে

১০.১১ Debugging নির্দেশিকা

- **Network Tab (DevTools):** প্রতিটি Request-এর ঠিকানা, Status, সময় এবং সাড়ার লেখা। "তথ্য আসছে না" সমস্যায় প্রথম প্রশ্ন: Request আদৌ গেছে কি, এবং Status কত।
- **Throttling:** Network Tab-এর "No throttling" তালিকা থেকে "Slow 4G" বা "3G" বেছে নিলে ধীর Network-এর অভিজ্ঞতা পাওয়া যায়। Loading অবস্থা এবং Race Condition এভাবেই চোখে পড়ে।
- **Offline:** একই তালিকার "Offline" দিয়ে Network বিচ্ছিন্নতার আচরণ দেখুন।
- **Async Stack Trace:** DevTools এবং Node.js দুটিই Error-এর Stack-এ `await`-এর আগের Function-গুলোও দেখায়; Stack-এর প্রতিটি লাইন পড়ুন।
- **Breakpoint `await`-এর পরের লাইনে** দিলে ফল আসার মুহূর্তে থামে, এবং পাওয়া মান দেখা যায়।

তথ্য প্রত্যাশামতো না এলে ক্রমে দেখুন:

১. Console-এ লাল Error বা "Uncaught (in promise)" আছে কি না।
২. Network Tab-এ Request গেছে কি না, Status এবং সাড়ার লেখা কী।
৩. যে Function Promise ফেরত দেয়, তার আগে `await` আছে কি না।
৪. `response.ok` যাচাই এবং `response.json()`-এর আগে `await` আছে কি না।
৫. ফল একাধিকবার আসছে, বা পুরোনো ফল নতুনটির ওপরে বসছে কি না।

১০.১২ পেশাদার অভ্যাস

- **নতুন Code-এ `async` / `await`,** এবং প্রতিটি `await`-এর জন্য ত্রুটি কোথায় ধরা হবে তা ঠিক করে রাখুন।
- **প্রতিটি Network Request-এ সময়সীমা,** এবং ব্যবহারকারী-চালিত Request-এ বাতিলের ব্যবস্থা।
- **তিনটি অবস্থা দেখান:** Loading, ফলাফল, এবং ত্রুটি—এবং ত্রুটি থেকে বেরোনোর উপায় (আবার চেষ্টার Button)।
- **ব্যবহারকারীর ভাষায় বার্তা:** "HTTP 503" নয়, "Server-এ সমস্যা হয়েছে। একটু পরে আবার চেষ্টা করুন।" প্রযুক্তিগত বিবরণ Console-এ।

- **Server-এর তথ্যকে অবিশ্বাস করুন:** গঠন যাচাই করুন, অপ্রত্যাশিত তথ্যে ভেঙে পড়বেন না (অধ্যায় ৯-এর `parseStored`-এর মতো)।
- **Network-কে Inject করুন:** Function-কে `fetch`-এর বদলে একটি `fetchImpl` দেওয়া যায় এমনভাবে লিখুন, যাতে Test-এ কৃত্রিম সাড়া দেওয়া যায়।
- **স্বাধীন কাজ একসঙ্গে,** নির্ভরশীল কাজ ক্রমে।

১০.১৩ Project: User Directory API Explorer

একটি Web Page, যা একটি Server থেকে User-দের তালিকা আনে, এবং নাম, Email বা শহর দিয়ে খোঁজা ও সাজানোর সুযোগ দেয়। মূল লক্ষ্য Network-এর সব বাস্তব পরিস্থিতি সঠিকভাবে সামলানো: ধীর Server, ত্রুটি, সাময়িক বিভ্রাট, ভাঙা তথ্য এবং দ্রুত পরপর Request।

Project-এর চাহিদা

- Page খুললে User-দের তালিকা আনা; চলাকালীন "লোড হচ্ছে..." দেখানো।
- খোঁজা (নাম, Email, শহর) এবং সাজানো (নাম, Email, শহর) Server-এ নতুন Request ছাড়াই।
- প্রতিটি Request-এ সময়সীমা; সাময়িক ত্রুটিতে (Network, সময়সীমা, `5xx`) দুবার পর্যন্ত Backoff-সহ আবার চেষ্টা; `4xx`-এ নয়।
- ত্রুটিতে ব্যবহারকারীর ভাষায় বার্তা, এবং "আবার লোড করুন" দিয়ে নতুন চেষ্টা।
- "আবার লোড করুন" চলমান Request বাতিল করে নতুনটি শুরু করবে; পুরোনো সাড়া কখনো নতুনটির ওপরে বসবে না।
- Server-এর তথ্য যাচাই; ভুল গঠনের User বাদ।
- সব লেখা `textContent` দিয়ে; বার্তা `aria-live`-এ; Loading-এর সময় তালিকায় `aria-busy`।
- Test-এ কোনো Public API নয়: Unit Test কৃত্রিম `fetch` দিয়ে, Integration ও Browser Test একটি Local Server দিয়ে।

File-এর কাঠামো

Folder-এর কাঠামো

```
user-directory-explorer/
├── index.html
├── style.css
├── directory.js            তথ্য আনা ও সাজানোর নিয়ম; DOM নেই
├── app.js                  Page, Event ও অবস্থা
├── README.md
└── tests/
    ├── load-script.js
    ├── directory.test.js   কৃত্রিম fetch এবং Local Server দিয়ে Test
    ├── mock-server.js      পরীক্ষার জন্য নিয়ন্ত্রিত Local Server
    ├── cdp.mjs
    └── browser-check.mjs   আসল Chrome-এ Page-এর Test
```

index.html ও style.css

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>User Directory</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>User Directory</h1>

    <div class="controls">
      <label>খুঁজুন <input id="search" type="search"></label>
      <label>সাজান
        <select id="sort">
          <option value="name">নাম</option>
          <option value="email">Email</option>
          <option value="city">শহর</option>
        </select>
      </label>
      <button id="reload" type="button">আবার লোড করুন</button>
    </div>

    <p id="status" role="status" aria-live="polite"></p>
    <ul id="users" aria-busy="false"></ul>
  </main>

  <script src="directory.js"></script>
```

```
  <script src="app.js"></script>
</body>
</html>
```

style.css · CSS চলবে: Browser

```
body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
  max-width: 720px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

.controls {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

input,
select,
button {
  font: inherit;
  padding: 6px 10px;
}

#users {
  padding: 0;
  list-style: none;
}

#users li {
  padding: 10px 0;
  border-bottom: 1px solid #e3e7ef;
}

#users[aria-busy='true'] {
  opacity: 0.5;
}
```

শেষ নিয়মটি লক্ষ করুন: Loading-এর সময় তালিকা ঝাপসা হয়, এবং তা নির্ধারিত হয় `aria-busy` Attribute দিয়ে, যা Screen Reader-কেও একই তথ্য দেয়। চেহারা ও Accessibility একই উৎস থেকে।

directory.js

তথ্য আনা, যাচাই, খোঁজা ও সাজানোর সব নিয়ম; DOM-এর কোনো উল্লেখ নেই। দুই অংশ মিলে সম্পূর্ণ File।

directory.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
'use strict';

const USERS_URL = 'https://jsonplaceholder.typicode.com/users';
const SORT_KEYS = ['name', 'email', 'city'];

// API-র একটি User থেকে Page-এর প্রয়োজনীয় অংশ; ভুল গঠনে null।
function normalizeUser(raw) {
  if (!Number.isInteger(raw?.id) || typeof raw.name !== 'string') {
    return null;
  }

  return {
    id: raw.id,
    name: raw.name.trim(),
    email: typeof raw.email === 'string' ? raw.email.trim() : '',
    city: typeof raw.address?.city === 'string' ? raw.address.city : 'অজানা',
  };
}

function filterUsers(users, query = '') {
  const needle = query.trim().toLowerCase();

  return users.filter((user) =>
    [user.name, user.email, user.city].some((text) =>
      text.toLowerCase().includes(needle)));
}

function sortUsers(users, key = 'name') {
  const field = SORT_KEYS.includes(key) ? key : 'name';
  return [...users].sort((a, b) => a[field].localeCompare(b[field]));
}

// ms মিলিসেকেন্ড অপেক্ষা; signal বাতিল হলে সঙ্গে সঙ্গে rejected।
function wait(ms, signal) {
  return new Promise((resolve, reject) => {
    if (signal?.aborted) {
      reject(signal.reason);
      return;
    }
```

```
    const timer = setTimeout(resolve, ms);
    signal?.addEventListener('abort', () => {
      clearTimeout(timer);
      reject(signal.reason);
    }, { once: true });
  });
}

// ...
```

অংশ ২: Network, Retry এবং বার্তা

directory.js · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser Console ও Node.js

```
// ...

// একবার চেষ্টা: সময়সীমা, বাতিল এবং HTTP ত্রুটি সামলায়।
async function fetchJson(url, options = {}) {
  const { fetchImpl = fetch, signal, timeoutMs = 5000 } = options;
  const timeout = AbortSignal.timeout(timeoutMs);
  const combined = signal ? AbortSignal.any([signal, timeout]) : timeout;
  const headers = { Accept: 'application/json' };
  const response = await fetchImpl(url, { signal: combined, headers });

  if (!response.ok) {
    const error = new Error(`HTTP ${response.status}`);
    error.status = response.status;
    throw error;
  }
  return response.json();
}

// সাময়িক ত্রুটি: Network বিচ্ছিন্ন, সময়সীমা পার, অথবা Server-এর 5xx।
const isTemporary = (error) =>
  error.name === 'TypeError' || error.name === 'TimeoutError' ||
  error.status >= 500;

async function fetchWithRetry(url, options = {}) {
  const { retries = 2, backoffMs = 300, signal } = options;

  for (let attempt = 0; ; attempt += 1) {
    try {
      return await fetchJson(url, options);
    } catch (error) {
      if (attempt >= retries || signal?.aborted || !isTemporary(error)) {
        throw error;
      }
      await wait(backoffMs * 2 ** attempt, signal);
    }
```

```
  }
}

async function loadUsers(options = {}) {
  const data = await fetchWithRetry(options.url ?? USERS_URL, options);

  if (!Array.isArray(data)) {
    throw new Error('Server থেকে অপ্রত্যাশিত তথ্য এসেছে।');
  }
  return data.map(normalizeUser).filter(Boolean);
}

// প্রযুক্তিগত ত্রুটি থেকে ব্যবহারকারীর ভাষায় বার্তা।
function errorMessage(error) {
  if (error.name === 'TimeoutError') {
    return 'Server সাড়া দিতে দেরি করছে। একটু পরে আবার চেষ্টা করুন।';
  }
  if (error.status >= 500) {
    return 'Server-এ সমস্যা হয়েছে। একটু পরে আবার চেষ্টা করুন।';
  }
  if (error.status === 404) {
    return 'তথ্যের ঠিকানা পাওয়া যায়নি।';
  }
  if (error.name === 'TypeError') {
    return 'Network-এ সংযোগ নেই। সংযোগ দেখে আবার চেষ্টা করুন।';
  }
  return 'তথ্য আনা যায়নি। আবার চেষ্টা করুন।';
}
```

- **`fetchImpl = fetch`**: স্বাভাবিক অবস্থায় আসল `fetch`, Test-এ কৃত্রিম একটি (পেশাদার অভ্যাস: Network-কে Inject করা)।
- **`AbortSignal.any([signal, timeout])`** দুটি Signal জোড়া দেয়: ব্যবহারকারী বাতিল করলে, অথবা সময় শেষ হলে—যেটি আগে ঘটে।
- **`fetchWithRetry`** অনুচ্ছেদ ১০.৯-এর Retry, তিনটি শর্তসহ: চেষ্টা শেষ, ব্যবহারকারী বাতিল করেছেন, অথবা ত্রুটি সাময়িক নয়—যেকোনোটি হলে আর চেষ্টা নয়। Backoff-এর অপেক্ষাও `signal` মানে, তাই বাতিল করলে অপেক্ষার মাঝেই থামে।
- **`loadUsers`** Server-এর তথ্য যাচাই করে: Array না হলে ত্রুটি, আর `normalizeUser` থেকে `null` আসা User-গুলো `filter(Boolean)` দিয়ে বাদ।
- **`errorMessage`** ক্রম গুরুত্বপূর্ণ: Network ত্রুটির নাম `TypeError`, কিন্তু Program-এর ভুলও `TypeError` হতে পারে। তাই নির্দিষ্ট ক্ষেত্রগুলো আগে, সাধারণ বার্তা শেষে।

app.js

Page-এর সঙ্গে সব যোগাযোগ এবং তিনটি অবস্থা (Loading, ফলাফল, ত্রুটি)। দুই অংশ মিলে সম্পূর্ণ File।

app.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
'use strict';

// ?api= ও ?timeout= শুধু পরীক্ষার জন্য; স্বাভাবিক ব্যবহারে আসল API।
const params = new URLSearchParams(location.search);
const API_URL = params.get('api') ?? USERS_URL;
const TIMEOUT_MS = Number(params.get('timeout') ?? 5000);

const search = document.querySelector('#search');
const sortSelect = document.querySelector('#sort');
const reloadButton = document.querySelector('#reload');
const statusView = document.querySelector('#status');
const list = document.querySelector('#users');

let users = [];
let controller = null;

function userCard(user) {
  const item = document.createElement('li');
  const name = document.createElement('h2');
  const details = document.createElement('p');

  name.textContent = user.name;
  details.textContent = `${user.email} · ${user.city}`;
  item.append(name, details);
  return item;
}

function render() {
  const shown = sortUsers(filterUsers(users, search.value), sortSelect.value);
  list.replaceChildren(...shown.map(userCard));
  statusView.textContent = users.length === 0
    ? 'কোনো User পাওয়া যায়নি।'
    : `${users.length} জনের মধ্যে ${shown.length} জন দেখানো হচ্ছে।`;
}

// ...
```

app.js · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

function showLoading(isLoading) {
  list.setAttribute('aria-busy', String(isLoading));
  if (isLoading) {
    statusView.textContent = 'লোড হচ্ছে...';
  }
}
```

```
async function refresh() {
  controller?.abort();
  const current = new AbortController();
  controller = current;
  showLoading(true);

  try {
    const loaded = await loadUsers({
      url: API_URL,
      signal: current.signal,
      timeoutMs: TIMEOUT_MS,
    });

    if (controller !== current) {
      return;
    }
    users = loaded;
    render();
  } catch (error) {
    if (controller !== current) {
      return;
    }
    users = [];
    list.replaceChildren();
    statusView.textContent = errorMessage(error);
    console.warn('User আনা যায়নি:', error);
  } finally {
    if (controller === current) {
      showLoading(false);
    }
  }
}

search.addEventListener('input', render);
sortSelect.addEventListener('change', render);
reloadButton.addEventListener('click', refresh);

refresh();
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **`refresh` -এর শুরু।** আগের Request চললে `controller?.abort()` সেটি বাতিল করে। তারপর এই Request-এর নিজস্ব Controller তৈরি হয় এবং `controller` -এ বসে।
- **"আমি কি এখনো সর্বশেষ?"** `await` -এর পরে `controller !== current` মানে এর মধ্যে নতুন Request শুরু হয়েছে; তখন এই ফল বা ত্রুটি কিছুই পর্দায় যায় না। এটি অনুচ্ছেদ ১০.৮-এর `ticket` -এর কাজটিই করে, আলাদা Counter ছাড়া। বাতিল হওয়া Request-এর `AbortError` -ও এই পথেই নিঃশব্দে বাদ পড়ে।

- `finally` সফল হোক বা ব্যর্থ, Loading বন্ধ করে; তবে শুধু সর্বশেষ Request-এর জন্য, যাতে পুরোনো Request নতুনটির Loading অবস্থা মুছে না দেয়।
- **ত্রুটি।** ব্যবহারকারী পান `errorMessage`-এর বোধগম্য বার্তা, ডেভেলপার পান Console-এ পুরো ত্রুটি। পুরোনো তালিকা মুছে দেওয়া হয়, যাতে ত্রুটির বার্তার পাশে বাসি তথ্য না থাকে।
- **খোঁজা ও সাজানো** Network ছোঁয় না; আগে আনা `users` থেকে শুধু `render`।
- `?api=` **এবং** `?timeout=` স্বাভাবিক ব্যবহারে থাকে না। Browser Test এগুলো দিয়ে Page-কে আসল API-র বদলে Local Server-এর দিকে পাঠায়।

চালানো এবং প্রত্যাশিত আচরণ

Browser-এর নিরাপত্তা-নিয়মের কারণে `file://` থেকে খোলা Page কিছু Server-এ Request পাঠাতে পারে না। তাই Project Folder থেকে একটি Local Server চালান, তারপর Browser-এ `http://localhost:8000` খুলুন (Internet সংযোগ লাগবে):

Terminal

```
python -m http.server 8000
```

Test চালাতে (Internet লাগে না):

Terminal

```
node --test
node tests/browser-check.mjs
```

পরিস্থিতি	যা হওয়ার কথা
Page খোলা	"লোড হচ্ছে…", তারপর তালিকা এবং "10 জনের মধ্যে 10 জন দেখানো হচ্ছে।"
খুঁজুন বা সাজান	সঙ্গে সঙ্গে তালিকা বদলায়; Network-এ নতুন Request যায় না
DevTools-এ Offline, তারপর "আবার লোড করুন"	কয়েক মুহূর্ত পর Network-এর বার্তা
Server `503` দুবার, তারপর সফল	Backoff-সহ আবার চেষ্টা; শেষে তালিকা
Server `404`	সঙ্গে সঙ্গে বার্তা; আবার চেষ্টা হয় না
Server সাড়া দেয় না	সময়সীমা পার হলে দেরির বার্তা
Loading চলাকালীন আবার "আবার লোড করুন"	পুরোনো Request বাতিল; শুধু নতুনটির ফল

Companion Source Code-এর `directory.test.js` কৃত্রিম `fetch` এবং একটি Local Server দিয়ে ১৮টি Test চালায়; `browser-check.mjs` আসল Chrome-এ Page খুলে ওপরের সারণির Local Server-এর পরিস্থিতিগুলোসহ ১২টি আচরণ যাচাই করে। কোনো Test Public API ব্যবহার করে না, তাই ফল প্রতিবার একই।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`loadUsers is not defined`	`directory.js` লোড হয়নি বা ক্রম উল্টো	Script-এর ক্রম
"লোড হচ্ছে..." থেকে নড়ে না	`finally` নেই, বা Promise কখনো শেষ হয় না	`finally`; সময়সীমা
CORS ত্রুটি Console-এ	`file://` থেকে খোলা, বা Server অনুমতি দেয় না	Local Server দিয়ে চালান
`404`-এ বারবার চেষ্টা	Retry সব ত্রুটিতে	`isTemporary`
দ্রুত দুবার Click-এ পুরোনো তালিকা ফেরে	সর্বশেষ Request যাচাই নেই	`controller !== current`
তালিকায় `undefined`	API-র গঠন বদলেছে	`normalizeUser` এবং যাচাই

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। JSONPlaceholder একটি Demo API: তথ্য কাল্পনিক, এবং তার প্রাপ্যতার নিশ্চয়তা নেই। Page-এ Pagination নেই; হাজারো User-এর জন্য Server-এ খোঁজা ও ভাগে ভাগে আনা লাগত। একবার আনা তথ্য Cache করা হয় না।

১০.১৪ অনুশীলনী

প্রাথমিক স্তর

১. না চালিয়ে বলুন Output-এর ক্রম কী হবে: `console.log('ক')`, `setTimeout` দিয়ে `'খ'`, `Promise.resolve().then` দিয়ে `'গ'`, `console.log('ঘ')`।

২. `wait(ms)` নামে একটি Function লিখুন, যা `ms` মিলিসেকেন্ড পরে fulfilled হয়। এটি দিয়ে `'১'` দেখিয়ে ৩০০ মিলিসেকেন্ড পরে `'২'` দেখান।

৩. `getUser().then((user) => getOrders(user.id)).then((orders) => console.log(orders.length))` Chain-টি `async` / `await` দিয়ে লিখুন।

৪. `const data = fetchData(); console.log(data.length);` এখানে `fetchData` একটি `async` Function। কেন `undefined` দেখায়? ঠিক করুন।

৫. একটি `async` Function লিখুন যা ব্যর্থ হয়, এবং `try` / `catch` / `finally` দিয়ে ত্রুটি ও "Loading বন্ধ" দেখান।

মধ্যম স্তর

৬. তিনটি পণ্য একসঙ্গে আনুন (প্রতিটি ১০০ মিলিসেকেন্ডের কৃত্রিম অপেক্ষা), তারপর মোট দাম দেখান।

৭. তিনটি Request-এর একটি ব্যর্থ হলেও বাকিদের ফল দেখান; কোনটি ব্যর্থ তাও জানান।

৮. `withTimeout(promise, ms)` লিখুন, যা `ms`-এর মধ্যে শেষ না হলে একটি ত্রুটি দিয়ে rejected হয়।

৯. কৃত্রিম `fetch` দিয়ে `getJson(url)` পরীক্ষা করুন তিনটি ক্ষেত্রে: সফল, `404`, এবং ভাঙা JSON।

চ্যালেঞ্জ স্তর

১০. `retryServerErrors(task, retries)` লিখুন, যা শুধু `5xx` Status-এর ত্রুটিতে আবার চেষ্টা করে, `4xx`-এ নয়।

১১. `runLimited(tasks, limit)` লিখুন, যা অনেকগুলো কাজ চালায় কিন্তু একসঙ্গে `limit`-এর বেশি নয়।

১২. `debounce(fn, ms)` লিখুন: দ্রুত পরপর ডাকলে শুধু শেষ ডাকের `ms` মিলিসেকেন্ড পরে একবার `fn` চলবে। খোঁজার Box-এ প্রতিটি অক্ষরে Request না পাঠাতে এটি ব্যবহৃত হয়।

১০.১৫ অনুশীলনীর সমাধান

সমাধান ১

order.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('ক');
setTimeout(() => console.log('খ'), 0);
Promise.resolve().then(() => console.log('গ'));
console.log('ঘ');
```

Output

```
ক
ঘ
গ
খ
```

Synchronous লাইন দুটি আগে, তারপর Microtask (`গ`), শেষে Task (`খ`)।

সমাধান ২

wait.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

console.log('১');
wait(300).then(() => console.log('২'));
```

Output

```
১
২
```

সমাধান ৩

async-rewrite.js · JavaScript চলবে: Browser Console ও Node.js

```js
const getUser = async () => ({ id: 7, name: 'Sara' });
const getOrders = async (id) => [`${id}-A`, `${id}-B`];

async function showOrderCount() {
  const user = await getUser();
  const orders = await getOrders(user.id);
  console.log(`${user.name}: ${orders.length}টি Order`);
}

showOrderCount();
```

Output

```
Sara: 2টি Order
```

সমাধান ৪ — `fetchData()` একটি Promise ফেরত দেয়, Array নয়; Promise-এর `length` নেই। `await` দিতে হবে, এবং তার জন্য Code-কে একটি `async` Function-এ রাখতে হবে।

missing-await.js · JavaScript চলবে: Browser Console ও Node.js

```js
const fetchData = async () => [10, 20, 30];

async function main() {
  const data = await fetchData();
  console.log(data.length);
}

main();
```

Output

```
3
```

সমাধান ৫

try-finally.js · JavaScript — চলবে: Browser Console ও Node.js

```
async function loadReport() {
  throw new Error('Report পাওয়া যায়নি');
}

async function main() {
  try {
    await loadReport();
    console.log('এটি চলবে না');
  } catch (error) {
    console.log(`ধরা পড়েছে: ${error.message}`);
  } finally {
    console.log('Loading বন্ধ');
  }
}

main();
```

Output

```
ধরা পড়েছে: Report পাওয়া যায়নি
Loading বন্ধ
```

সমাধান ৬

all-products.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

const getProduct = (id, price) => wait(100, { id, price });

async function main() {
  const products = await Promise.all([
    getProduct('P1', 250),
    getProduct('P2', 120),
    getProduct('P3', 80),
  ]);
  const total = products.reduce((sum, product) => sum + product.price, 0);
  console.log(`${products.length}টি পণ্য, মোট ${total} টাকা`);
}

main();
```

Output

```
3টি পণ্য, মোট 450 টাকা
```

তিনটি অপেক্ষা একসঙ্গে চলেছে, তাই মোট সময় প্রায় ১০০ মিলিসেকেন্ড, ৩০০ নয়।

সমাধান ৭

all-settled.js · JavaScript — চলবে: Browser Console ও Node.js

```
function request(name, succeeds) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (succeeds) {
        resolve(`${name} সফল`);
      } else {
        reject(new Error(`${name} ব্যর্থ`));
      }
    }, 50);
  });
}

async function main() {
  const results = await Promise.allSettled([
    request('Profile', true),
    request('Orders', false),
    request('Settings', true),
  ]);

  for (const result of results) {
    const ok = result.status === 'fulfilled';
    console.log(ok ? result.value : result.reason.message);
  }
}

main();
```

Output

```
Profile সফল
Orders ব্যর্থ
Settings সফল
```

সমাধান ৮

with-timeout.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

function withTimeout(promise, ms) {
  let timer;
  const timeout = new Promise((_, reject) => {
    timer = setTimeout(() => reject(new Error(`${ms} ms পার হয়েছে`)), ms);
  });
  return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
}

async function main() {
  console.log(await withTimeout(wait(50, 'সময়মতো এসেছে'), 200));

  try {
    await withTimeout(wait(500, 'দেরিতে'), 200);
  } catch (error) {
    console.log(error.message);
  }
}

main();
```

Output

```
সময়মতো এসেছে
200 ms পার হয়েছে
```

`Promise.race` ধীর কাজটিকে থামায় না, শুধু তার জন্য অপেক্ষা বন্ধ করে। কাজটিই থামাতে হলে `AbortController` লাগে, যেমন Project-এ।

সমাধান ৯

get-json.js · JavaScript — চলবে: Browser Console ও Node.js

```
const responses = {
  '/ok': () => new Response('{"total":5}', { status: 200 }),
  '/missing': () => new Response('', { status: 404 }),
  '/broken': () => new Response('{oops', { status: 200 }),
};
const fakeFetch = async (url) => responses[url]();

async function getJson(url) {
```

```
  const response = await fakeFetch(url);

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return response.json();
}

async function main() {
  for (const url of Object.keys(responses)) {
    try {
      const data = await getJson(url);
      console.log(`${url}: total ${data.total}`);
    } catch (error) {
      const reason = error instanceof SyntaxError ? 'ভাঙা JSON' : error.message;
      console.log(`${url}: ${reason}`);
    }
  }
}

main();
```

Output

```
/ok: total 5
/missing: HTTP 404
/broken: ভাঙা JSON
```

ভাঙা JSON-এ `response.json()` একটি `SyntaxError` দিয়ে rejected হয়; Status `200` হলেও তথ্য ব্যবহারযোগ্য নয়।

সমাধান ১০

retry-5xx.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function retryServerErrors(task, retries = 2) {
  for (let attempt = 0; ; attempt += 1) {
    try {
      return await task();
    } catch (error) {
      if (!(error.status >= 500) || attempt >= retries) {
        throw error;
      }
      await wait(100 * 2 ** attempt);
    }
  }
```

```
}

function serverReplying(...statuses) {
  let call = 0;
  return async () => {
    const status = statuses[call];
    call += 1;
    if (status === 200) {
      return `সফল (${call} বারের চেষ্টায়)`;
    }
    const error = new Error(`HTTP ${status}`);
    error.status = status;
    throw error;
  };
}

async function main() {
  console.log(await retryServerErrors(serverReplying(503, 500, 200)));

  try {
    await retryServerErrors(serverReplying(404, 200));
  } catch (error) {
    console.log(`আবার চেষ্টা হয়নি: ${error.message}`);
  }
}

main();
```

Output

```
সফল (3 বারের চেষ্টায়)
আবার চেষ্টা হয়নি: HTTP 404
```

সমাধান ১১

run-limited.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function runLimited(tasks, limit) {
  const results = [];
  let next = 0;

  async function worker() {
    while (next < tasks.length) {
      const index = next;
      next += 1;
      results[index] = await tasks[index]();
```

```
    }
  }

  await Promise.all(Array.from({ length: limit }, worker));
  return results;
}

let running = 0;
const job = (name, ms) => async () => {
  running += 1;
  console.log(`শুরু ${name} (একসঙ্গে ${running})`);
  await wait(ms);
  running -= 1;
  return name;
};

const jobs = [job('ক', 120), job('খ', 50), job('গ', 50), job('ঘ', 50)];
runLimited(jobs, 2).then((names) => console.log(`শেষ: ${names.join(', ')}`));
```

Output

```
শুরু ক (একসঙ্গে 1)
শুরু খ (একসঙ্গে 2)
শুরু গ (একসঙ্গে 2)
শুরু ঘ (একসঙ্গে 2)
শেষ: ক, খ, গ, ঘ
```

দুটি `worker` একই তালিকা থেকে পালা করে কাজ নেয়। একটি কাজ শেষ হলেই সেই `worker` পরেরটি ধরে, তাই একসঙ্গে কখনো দুটির বেশি চলে না। ফলগুলো `results[index]`-এ রাখা হয়েছে, তাই শেষ হওয়ার ক্রম যা-ই হোক, ফল আসে মূল ক্রমে।

সমাধান ১২

debounce.js · JavaScript চলবে: Browser Console ও Node.js

```
function debounce(fn, ms) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), ms);
  };
}

const search = debounce((text) => console.log(`খোঁজা হলো: ${text}`), 300);

// ব্যবহারকারী দ্রুত লিখছেন: প্রতি 100 ms-এ একটি নতুন অক্ষর।
['J', 'Ja', 'Jav', 'Java'].forEach((text, index) => {
  setTimeout(() => search(text), index * 100);
});
setTimeout(() => search('JavaScript'), 900);
```

Output

```
খোঁজা হলো: Java
খোঁজা হলো: JavaScript
```

চারটি দ্রুত ডাকের মধ্যে শুধু শেষটি (`Java`) চলেছে; প্রতিটি নতুন ডাক আগের Timer বাতিল করেছে। `timer` Variable-টি Closure-এ থেকে যায় (অধ্যায় ৫)।

১০.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `setTimeout(fn, 0)` সম্পর্কে কোনটি সঠিক?

ক) `fn` সঙ্গে সঙ্গে চলে

খ) `fn` চলে বর্তমান কাজ এবং সব Microtask শেষ হওয়ার পর

গ) `fn` কখনো চলে না

ঘ) `fn` সব Promise-এর আগে চলে

প্রশ্ন ২. `console.log(1); Promise.resolve().then(() => console.log(2)); setTimeout(() => console.log(3)); console.log(4);`-এর Output?

ক) 1 2 3 4

খ) 1 4 3 2

গ) 1 4 2 3

ঘ) 1 2 4 3

প্রশ্ন ৩. একটি Promise-এর অবস্থা কোনটি নয়?

ক) pending

খ) fulfilled

গ) rejected

ঘ) cancelled

প্রশ্ন ৪. একটি `async` Function সব সময় কী ফেরত দেয়?

ক) Promise

খ) `undefined`

গ) `return`-এ লেখা মানটি

ঘ) একটি Callback

প্রশ্ন ৫. Server `404` পাঠালে `fetch`-এর Promise কী হয়?

ক) rejected হয়

খ) fulfilled হয়; `response.ok` হয় `false`

গ) pending থেকে যায়

ঘ) `null` দিয়ে fulfilled হয়

প্রশ্ন ৬. তিনটি স্বাধীন Request-এর সবগুলোর ফল একসঙ্গে দরকার, এবং একটি ব্যর্থ হলে পুরো কাজ ব্যর্থ। কোনটি?

ক) `Promise.race`

খ) `Promise.any`

গ) `Promise.all`

ঘ) পরপর তিনটি `await`

প্রশ্ন ৭. একটি চলমান `fetch` বাতিল করতে কী লাগে?

ক) `clearTimeout`

খ) `AbortController`

গ) `Promise.reject`

ঘ) `break`

প্রশ্ন ৮. `ids.forEach(async (id) => { await save(id); }); console.log('শেষ');` — `'শেষ'` কখন ছাপা হয়?

ক) সব `save` শেষ হওয়ার পর

খ) প্রথম `save` শেষ হওয়ার পর

গ) কখনোই না

ঘ) কোনো `save` শেষ হওয়ার আগেই

প্রশ্ন ৯. কোন ত্রুটিতে আবার চেষ্টা (Retry) করা উচিত নয়?

ক) `404 Not Found`

খ) `503 Service Unavailable`

গ) সময়সীমা পার

ঘ) ক্ষণিকের Network বিচ্ছিন্নতা

প্রশ্ন ১০. দ্রুত পরপর দুটি খোঁজায় পুরোনো সাড়া দেরিতে এসে নতুন ফলের ওপরে বসল। এর নাম?

ক) Callback Hell

খ) Race Condition

গ) Memory Leak

ঘ) Hoisting

১০.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ঘ · ৪–ক · ৫–খ · ৬–গ · ৭–খ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Timer-এর Callback একটি Task; Stack খালি হওয়া এবং সব Microtask-এর পরে চলে।
২	গ	Synchronous 1 ও 4, তারপর Microtask 2, শেষে Task 3।
৩	ঘ	Promise-এর অবস্থা তিনটি; বাতিল করলে সেটি `AbortError` দিয়ে rejected হয়।
৪	ক	`return 42` লিখলেও ফল `42` দিয়ে fulfilled একটি Promise।

প্রশ্ন	উত্তর	ব্যাখ্যা
৫	খ	সাড়া এসেছে, তাই fulfilled; HTTP ত্রুটি `response.ok` দিয়ে যাচাই করতে হয়।
৬	গ	`Promise.all` একসঙ্গে চালায়, এবং একটি ব্যর্থ হলেই ব্যর্থ হয়।
৭	খ	Controller-এর `signal` `fetch`-কে দিয়ে `abort()` ডাকতে হয়।
৮	ঘ	`forEach` Callback-এর Promise-এর জন্য অপেক্ষা করে না।
৯	ক	ঠিকানা নেই—আবার পাঠালেও একই ফল; Retry শুধু সাময়িক ত্রুটিতে।
১০	খ	ফল নির্ভর করছে কোন কাজ আগে শেষ হয় তার ওপর।

১০.১৮ অধ্যায়ের সারসংক্ষেপ

- JavaScript একসময়ে একটি কাজ করে; অপেক্ষার কাজ Host সামলায়, ফল আসে Queue হয়ে।
- Event Loop: বর্তমান কাজ, তারপর সব Microtask (Promise), তারপর একটি Task (Timer, Event)।
- Callback প্রথম পদ্ধতি; গভীর Nesting-এ পড়া ও ত্রুটি সামলানো কঠিন।
- Promise-এর তিন অবস্থা; `then` Chain করে, একটি `catch` সব ধাপের ত্রুটি ধরে, `finally` সব ক্ষেত্রে চলে।
- `async` Function সব সময় Promise ফেরত দেয়; `await` শুধু সেই Function থামায়; ত্রুটি `try` / `catch`-এ।
- `fetch` HTTP ত্রুটিতে fulfilled হয়; `response.ok` যাচাই বাধ্যতামূলক; Body পড়তে আরেকটি `await`।
- স্বাধীন কাজ `Promise.all` দিয়ে একসঙ্গে; ব্যর্থতার প্রয়োজন অনুযায়ী `allSettled`, `race`, `any`।
- `AbortController` বাতিল করে, `AbortSignal.timeout` সময়সীমা দেয়; সর্বশেষ Request-এর ফলই গ্রহণ করে Race Condition এড়ান।
- Retry শুধু সাময়িক ত্রুটিতে, সীমিত সংখ্যায়, Exponential Backoff-সহ।
- Network-কে Inject করে কৃত্রিম সাড়া দিয়ে Test করুন; Public API-র ওপর Test নির্ভর করবেন না।

পরের অধ্যায়ে Error Handling: ত্রুটির ধরন, নিজের Error তৈরি, প্রত্যাশিত ও অপ্রত্যাশিত ত্রুটির পার্থক্য, এবং Program-কে ত্রুটির মধ্যেও নির্ভরযোগ্য রাখার কৌশল।

১০.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Asynchronous	অপেক্ষার কাজ পেছনে চলে, Program এগিয়ে যায়, ফল পরে আসে।
Call Stack	এই মুহূর্তে চলমান Function-গুলোর স্তূপ।
Event Loop	Stack খালি হলে Queue থেকে পরের কাজ এনে চালানোর ব্যবস্থা।
Task	Timer, Event-এর মতো Callback; একবারে একটি চলে।

পরিভাষা	অর্থ
Microtask	Promise-এর Callback; প্রতিটি Task-এর পরে সবগুলো চলে।
Callback	পরে ডাকার জন্য অন্য Function-কে দেওয়া Function।
Promise	একটি অপেক্ষার কাজের ভবিষ্যৎ ফলের প্রতিনিধি Object।
`async` / `await`	Promise-এর ওপর Synchronous-এর মতো দেখতে লেখার ধরন।
Response	`fetch`-এর ফল: Status, Header এবং Body।
AbortController	চলমান কাজ বাতিলের নিয়ন্ত্রক।
Race Condition	কাজ শেষের ক্রমের ওপর নির্ভর করে ভুল ফল।
Exponential Backoff	প্রতিবার ব্যর্থতার পর অপেক্ষা দ্বিগুণ করে আবার চেষ্টা।

১০.২০ আরও পড়ুন

- **MDN — Introducing asynchronous JavaScript:** `https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS/Introducing` — Asynchronous ধারণার ভূমিকা।
- **MDN — Using promises:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises` — Promise-এর বিস্তারিত নির্দেশিকা।
- **MDN — Using the Fetch API:** `https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch` — `fetch`, Response এবং ত্রুটি।
- **MDN — AbortController:** `https://developer.mozilla.org/en-US/docs/Web/API/AbortController` — বাতিল ও সময়সীমা।
- **Node.js — The Node.js Event Loop:** `https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick` — Node.js-এর Event Loop-এর বিবরণ।